

Document de veille technologique, d'audit et de recommandations

Le plan présenté dans ce document est proposé à titre d'exemple, illustrant la structure attendue dans le cadre d'un projet réel.

Dans le contexte de cet exercice, ne cherchez pas à détailler chaque section de manière excessive, cela prendrait trop de temps.

Considérez ce document comme un guide plutôt qu'une liste d'attentes strictes à respecter à la lettre.

1. Introduction.....	2
1.1 Objectif.....	2
1.2 Rappel du contexte.....	2
2. Rapport de veille technologique.....	2
2.1 Méthodologie.....	2
2.2 Technologies évaluées.....	2
2.2.1 <Technologie X>.....	2
2.3 Matrices comparatives.....	2
2.3.1 <Catégorie X>.....	3
3. Audit des processus de développement.....	3
3.1 Objectifs.....	3
3.2 Méthodologie.....	3
3.3. Processus audités.....	4
3.3.1 Gestion du code et intégration continue.....	4
3.3.2 Dépendances, tests et builds.....	4
3.3.3 Conteneurisation et transmission aux opérations.....	5
3.3.4 Qualité, sécurité et traçabilité.....	5
3.3.5 Configuration d'exécution et données.....	5
3.4 Synthèse.....	6
3.4.1 Analyse SWOT.....	6
4. Recommandations.....	8
4.1 Objectifs.....	8
4.2 Recommandations pour l'automatisation et la chaîne CI.....	8
4.2.1 Conserver et réutiliser les résultats de la CI.....	8
4.2.2 Renforcer progressivement les tests automatisés.....	8
4.2.3 Avancer les contrôles de sécurité dans la chaîne.....	9
4.2.4 Analyser continuellement la qualité du code.....	9
4.2.5 Automatiser et tracer la production des images.....	9
4.2.6 Rendre la configuration d'exécution portable.....	9
4.2.7 Définir la persistance, la sauvegarde et la restauration.....	10

4.2.8 Normaliser le workflow Git et les releases.....	10
4.2.9 Modulariser les opérations répétitives.....	10
4.2.10 Automatiser le retour d'information utile.....	11
5. Conclusion.....	11
5.1 Synthèse des constats.....	11
5.2 Prochaines étapes.....	11

1. Introduction

1.1 Objectif

Décrire l'objectif général du document et son importance pour le projet de l'entreprise.

1.2 Rappel du contexte

Fournir le contexte dans lequel cette veille technologique est réalisée, notamment les motivations à l'origine de celle-ci et les besoins spécifiques au contexte du projet.

2. Rapport de veille technologique

2.1 Méthodologie

Décrire les méthodes (essayer de modéliser l'approche qui a été suivie pour la recherche d'information) et les sources utilisées pour la veille technologique (ex: bibliographie des ressources documentaires, numériques ou physiques)

2.2 Technologies évaluées

Lister les différentes technologies évaluées dans le cadre de la veille technologique.

2.2.1 <Technologie X>

- **Description** : Rédiger une courte description de la technologie.
- **Catégorisation** : Identifier et assigner une catégorie à la technologie dans l'optique de réaliser la matrice comparative.
- **Fonctionnalités clés** : Lister les fonctionnalités clés associées à la technologie.
- **Cas d'usages** : Décrire les cas d'usages nominaux associés à la technologie.
- **Références** : Lister les références documentaires (URL, identifiant ISBN) qui ont servi à la réalisation de la veille pour cette technologie.

2.3 Matrices comparatives

Créer des matrices comparatives par catégorie listant les avantages/inconvénients des différentes technologies évaluées.

2.3.1 <Catégorie X>

Comparé à	Technologie A	Technologie B
Technologie A		Avantages - <i>Avantage B vs. A</i> Inconvénients - <i>Inconvénient B vs. A</i>
Technologie B	Avantages - <i>Avantage A vs. B</i> Inconvénients - <i>Inconvénient A vs. B</i>	

3. Audit des processus de développement

3.1 Objectifs

L'audit évalue l'efficacité de la chaîne d'intégration continue et des pratiques de développement actuellement utilisées pour MicroCRM. Il cherche à :

- reconstituer le fonctionnement réel depuis la modification du code jusqu'à la transmission d'une version aux opérations ;
- distinguer les pratiques décrites par les équipes de celles qui sont effectivement configurées dans le dépôt ;
- identifier les opérations manuelles, répétitions, points de friction et goulets d'étranglement ;
- mesurer l'état initial des tests, de la couverture, des builds et de la conteneurisation ;
- repérer les lacunes de qualité, de sécurité, de traçabilité et de portabilité ;
- rapprocher les besoins communs ou spécifiques des équipes Dev et Ops avant toute veille ou recommandation.

Cette analyse établit une baseline vérifiable. Elle servira à comparer l'existant avec la future chaîne CI full-stack, sans présumer à ce stade des outils ou solutions qui seront retenus.

3.2 Méthodologie

L'audit a été réalisé le 26 juillet 2026 à partir de sources de natures différentes :

- exigences officielles : contexte et consignes OpenClassrooms ;
- informations déclaratives : sondages des équipes Dev et Ops ;
- documentation existante : README et notes fournies ;
- observations techniques : arborescence Git, pipeline .gitlab-ci.yml, configurations Angular, Spring Boot et Docker ;
- mesures reproductibles : installation des dépendances, tests, couverture, builds, construction des images et smoke tests contrôlés.

Les déclarations des équipes ont été utilisées pour comprendre les usages et difficultés, puis confrontées au dépôt lorsque cela était possible. Une affirmation technique est

considérée comme confirmée lorsqu'elle renvoie à un fichier, une commande, un rapport ou un comportement reproduit. Les causes qui ne peuvent pas être prouvées restent présentées comme probables.

Les mesures ont été effectuées localement sur la branche dev au commit 526bd96cddd2903676988b56dfb2778667aa435, sans modifier le code. Elles constituent une baseline fonctionnelle et non une mesure de performance d'un runner GitLab ou d'un environnement de production. Aucun test de charge, déploiement AWS ou indicateur d'exploitation n'a été réalisé pendant cet audit.

3.3. Processus audités

3.3.1 Gestion du code et intégration continue

Description — MicroCRM est organisé en monorepo GitLab avec un frontend Angular et un backend Spring Boot. Le pipeline versionné comprend deux stages, test et build, et quatre jobs séparés pour les deux composants.

Forces — Le dépôt GitLab existe déjà ; les composants sont clairement séparés ; le frontend possède un fichier de verrouillage npm et le backend utilise le Gradle Wrapper. Les commandes principales sont présentes dans le README et dans la CI.

Faiblesses — Aucun workflow de branches, de merge request ou de validation avant intégration n'est documenté dans le dépôt. Le pipeline ne définit ni règles d'exécution selon les branches ou tags, ni cache, ni dépendances explicites entre jobs.

Risques — L'exécution peut être identique pour des événements qui devraient être traités différemment. L'absence de règles partagées et de traçabilité formalisée augmente le risque d'intégration incohérente.

Sources — README, .gitlab-ci.yml, état Git observé et sondage Dev.

3.3.2 Dépendances, tests et builds

Description — La CI installe les dépendances, lance les tests frontend et backend, puis reconstruit chaque composant dans des jobs distincts.

Forces — Les commandes actuelles réussissent : 8 tests frontend et 2 tests backend ont été reproduits sans échec. Les builds Angular et Gradle aboutissent. Une couverture frontend peut être générée localement.

Faiblesses — Les huit tests frontend vérifient surtout la création des composants et services ainsi que le titre de l'application. Le backend ne contient qu'un test de chargement du contexte et un test de repository. Aucun scénario HTTP ou CRUD complet n'est couvert. La couverture mesurée reste faible : 30,76 % des lignes et 9,52 % des branches. La CI ne publie ni rapport JUnit, ni couverture, ni artefact. L'installation npm est répétée et le build Gradle réexécute les deux tests déjà lancés par le job de test.

Risques — Des régressions fonctionnelles peuvent ne pas être détectées. Les répétitions allongent inutilement le pipeline, tandis que l'absence de rapports et d'artefacts empêche de conserver une preuve ou de réutiliser un build déjà validé.

Sources — Fichiers de tests, rapports Karma et Gradle, rapport de couverture, .gitlab-ci.yml et mesures D.4.

3.3.3 Conteneurisation et transmission aux opérations

Description — Le Dockerfile multi-stage peut produire trois cibles : frontend, backend et standalone. Selon les sondages, les Dev génèrent majoritairement les images manuellement, puis les Ops les contrôlent avec Trivy et les déploient manuellement sur un environnement de démonstration.

Forces — Docker constitue une interface commune aux deux équipes. Les trois cibles ont pu être construites localement. Le backend démarre et répond à l'API ; dans l'image standalone, Supervisor lance bien les processus frontend et backend.

Faiblesses — Le pipeline GitLab ne construit, ne publie et ne versionne aucune image. Aucun digest ou manifeste ne relie une version au commit qui l'a produite. Les images s'exécutent avec l'utilisateur root par défaut et ne déclarent aucun healthcheck. Le backend écoute sur 8080 alors que son image expose 4200. Le frontend contient bien ses fichiers statiques, mais le Caddyfile actuel déclenche une redirection HTTPS qui n'a pas permis de valider son smoke test.

Risques — Les manipulations manuelles favorisent les erreurs et les écarts de version. Une image peut parvenir tardivement aux contrôles de sécurité ou être difficile à relier au code source. Les incohérences de ports et de routage peuvent rendre le service indisponible.

Sources — Dockerfile, Caddyfile, supervisor.ini, .gitlab-ci.yml, sondages Dev/Ops et smoke tests D.4.

3.3.4 Qualité, sécurité et traçabilité

Description — Le dépôt actuel exécute uniquement les tests et builds applicatifs. Les Ops déclarent utiliser Trivy manuellement avant le déploiement.

Forces — Les équipes ont déjà identifié l'intérêt d'un contrôle des images. Les dépendances npm signalent les vulnérabilités pendant l'installation, ce qui rend une première observation possible.

Faiblesses — Aucun contrôle SonarQube, analyse de dépendances structurée, détection de secrets ou scan d'image n'est intégré au pipeline. L'installation npm observée pendant le build Docker signale 64 vulnérabilités, dont une critique et 46 élevées. Le détail des avis n'a pas été collecté pendant l'audit. Les sorties de qualité et de sécurité ne sont pas conservées dans GitLab.

Risques — Une anomalie peut être découverte seulement après la transmission de l'image aux Ops, comme l'illustre le retard déjà déclaré dans les sondages. L'absence de rapport lié au commit complique le suivi de la dette et la justification d'une acceptation ou d'un refus.

Sources — .gitlab-ci.yml, sortie npm ci du build Docker, sondages Dev/Ops et configurations du dépôt.

3.3.5 Configuration d'exécution et données

Description — Le frontend appelle une URL d'API définie dans le code, tandis que le backend utilise HSQLDB sans configuration de persistance externe. Le backend applique une configuration CORS permissive.

Forces — L'API démarre et répond correctement sur le port 8080. Les prérequis CORS testées pour GET et PUT sont acceptées. L'environnement embarqué permet une démonstration locale simple.

Faiblesses — L'adresse `http://localhost:8080` est intégrée jusque dans le bundle frontend et ne s'adapte pas à un autre environnement. La base HSQLDB perd la donnée ajoutée lorsque le conteneur est remplacé. Toutes les origines CORS sont autorisées et le comportement runtime autorise davantage de méthodes que la liste personnalisée visible dans le code.

Risques — Le frontend peut contacter la mauvaise machine après déploiement. Les données peuvent être perdues lors d'un remplacement de conteneur. Une configuration CORS trop large augmente inutilement la surface d'exposition.

Sources — `config.ts`, bundle Angular, `application.properties`, configuration Spring Data REST et tests contrôlés D.4.

3.4 Synthèse

L'existant fournit une base exploitable : le code est centralisé dans GitLab, le frontend et le backend sont séparés, les dépendances sont reproductibles, les tests et builds actuels réussissent, et les trois cibles Docker peuvent être construites.

Cette base ne constitue toutefois pas encore une chaîne de livraison complète. Le pipeline s'arrête aux tests et aux builds, répète certaines opérations et ne conserve ni rapports, ni artefacts, ni images. La profondeur fonctionnelle des tests et la couverture sont faibles. Les contrôles de qualité et de sécurité interviennent tardivement ou ne sont pas automatisés. Enfin, plusieurs écarts d'exécution ont été reproduits : port Docker incohérent, configuration frontend liée à localhost, frontend non validé au smoke test et données HSQLDB non persistantes.

Les sondages et l'observation du dépôt convergent donc sur quatre besoins : réduire les opérations manuelles, détecter plus tôt les problèmes de qualité et de sécurité, transmettre une image identifiable et vérifiée, et rendre la configuration portable entre les environnements. Ces besoins orienteront la veille technologique de l'étape suivante. Aucune solution n'est retenue par le présent audit.

Limites — Les durées ont été mesurées sur le poste local et non sur un runner GitLab. Aucun pipeline distant, test de charge, environnement AWS ou indicateur de production n'a été observé. Le détail des avis de sécurité npm reste à compléter dans un contexte réseau autorisé.

3.4.1 Analyse SWOT

La SWOT résume les constats précédents sans remplacer leurs preuves. Les forces et faiblesses concernent l'organisation et la chaîne actuelles ; les opportunités et menaces décrivent le contexte dans lequel elles doivent évoluer.

Forces — facteurs internes favorables

- Le code frontend et backend est déjà centralisé dans un dépôt GitLab et une CI minimale existe.
- Les dépendances disposent de mécanismes reproductibles : `package-lock` npm et Gradle

Wrapper.

- Les tests et builds actuels réussissent, ce qui fournit une baseline fonctionnelle mesurable.
- Docker constitue déjà une interface commune entre Dev et Ops ; les trois cibles d'image peuvent être construites.
- Les Ops pratiquent déjà un contrôle d'image avec Trivy, même s'il reste manuel.

Sources — dépôt MicroCRM, mesures de l'audit du 26 juillet 2026 et sondages Dev/Ops.

Faiblesses — facteurs internes défavorables

- La CI est limitée aux tests et builds : aucun rapport, artefact, cache, image, release ou déploiement n'est conservé ou automatisé.
- La profondeur des tests est faible et la couverture frontend mesurée atteint seulement 30,76 % des lignes et 9,52 % des branches.
- Certaines opérations sont répétées, notamment l'installation npm et l'exécution des tests backend pendant le build.
- Aucun contrôle automatisé de qualité, de dépendances, de secrets ou d'image n'est intégré ; 64 vulnérabilités npm sont signalées dans la baseline.
- La portabilité et la fiabilité sont limitées par l'URL localhost, le port Docker incohérent, la configuration Caddy, l'absence de healthcheck et la base non persistante.
- Le workflow de branches, la traçabilité des versions et les responsabilités de livraison ne sont pas documentés.

Sources — .gitlab-ci.yml, Dockerfile, configurations applicatives et mesures de l'audit du 26 juillet 2026.

Opportunités — facteurs contextuels favorables

- Les besoins Dev et Ops convergent sur la réduction des manipulations manuelles et la détection plus précoce des problèmes.
- L'usage déjà commun de GitLab et Docker réduit l'effort nécessaire pour normaliser les échanges entre les équipes.
- Les outils modernes de CI, de qualité, de sécurité et de registre peuvent être évalués comme extensions de l'existant, sans remplacer toute la chaîne.
- La future CI full-stack offre l'occasion de produire une preuve unique reliant code, tests, contrôles et image livrable.

Sources — contexte OpenClassrooms, sondages Dev/Ops et contraintes techniques observées. Les options concrètes seront comparées pendant la veille.

Menaces — facteurs contextuels défavorables

- Les vulnérabilités des dépendances et images évoluent dans le temps et peuvent apparaître après une première validation.
- Les tags flottants et les dépôts externes de paquets ou d'images peuvent modifier un build ou devenir temporairement indisponibles.
- Un contrôle de sécurité tardif peut encore retarder la livraison et augmenter les allers-retours entre Dev et Ops.
- Les écarts de vocabulaire et de cible déclarée — staging ou démonstration, HSQLDB ou PostgreSQL — peuvent conduire à une solution inadaptée si les besoins ne sont pas

clarifiés.

Sources — sondages Dev/Ops, configurations versionnées et incident de téléchargement observé pendant l'audit.

Conclusion SWOT — Les forces permettent d'améliorer progressivement l'existant. Les faiblesses et menaces justifient une veille ciblée sur l'automatisation, la qualité, la sécurité, la traçabilité et la portabilité. La SWOT ne sélectionne aucun outil : elle définit les besoins auxquels les options étudiées devront répondre.

4. Recommandations

4.1 Objectifs

Les recommandations initiales ont pour objectif de transformer les faiblesses et risques observés pendant l'audit en capacités concrètes à obtenir : une intégration continue plus fiable, des retours plus rapides, des contrôles de qualité et de sécurité plus précoces, une livraison traçable et une configuration portable.

À ce stade, elles constituent un backlog de besoins. Elles ne sélectionnent pas encore les solutions techniques : les options, leur compatibilité, leur coût et leur charge de maintenance seront comparés pendant la veille technologique.

4.2 Recommandations pour l'automatisation et la chaîne CI

Le backlog ci-dessous est issu des constats de l'audit. Les priorités tiennent compte de la sécurité, de la fiabilité, de la rapidité, de la maintenabilité et du coût. « P1 » désigne les travaux de la partie 1 ; « P2 » désigne les réalisations dépendant de l'environnement AWS.

4.2.1 Conserver et réutiliser les résultats de la CI

Constat — La CI ne publie ni rapports, ni couverture, ni artefacts. Les dépendances et certains tests ou builds sont exécutés plusieurs fois.

Cause — Les jobs sont indépendants et ne transmettent pas leurs sorties aux étapes suivantes.

Action possible — Produire des rapports standardisés, conserver les livrables utiles et réutiliser un résultat déjà validé au lieu de le reconstruire.

Bénéfices attendus — Obtenir une preuve liée au pipeline, réduire les répétitions, accélérer les retours et limiter la consommation de ressources CI.

Priorité et périmètre — Élevée — P1 — fiabilité, rapidité et coût.

4.2.2 Renforcer progressivement les tests automatisés

Constat — Les tests réussissent, mais couvrent peu les comportements métier ; aucun scénario HTTP ou CRUD complet n'est présent et la couverture frontend mesurée reste faible.

Cause — La suite actuelle constitue surtout une baseline technique et aucun critère de progression n'est exploité dans la CI.

Action possible — Ajouter des tests ciblés sur les appels HTTP, les opérations CRUD, les scripts et le démarrage des images ; publier leurs rapports et établir une baseline avant de

fixer des seuils.

Bénéfices attendus — Détecter davantage de régressions avant l'intégration et rendre l'évolution de la qualité mesurable.

Priorité et périmètre — Élevée — P1 — fiabilité et qualité.

4.2.3 Avancer les contrôles de sécurité dans la chaîne

Constat — Le contrôle des images est déclaré comme manuel et tardif ; aucun scan de dépendances, de secrets ou d'images n'est intégré à la CI, alors que la baseline npm signale 64 vulnérabilités.

Cause — Les contrôles sont effectués après la transmission aux Ops ou se limitent aux avertissements visibles pendant l'installation.

Action possible — Automatiser les contrôles des dépendances, secrets et images aux moments adaptés, conserver les rapports et définir une politique de traitement après qualification de la dette existante.

Bénéfices attendus — Détecter les risques plus tôt, réduire les allers-retours Dev/Ops et justifier les anomalies bloquées, corrigées ou temporairement acceptées.

Priorité et périmètre — Critique — P1 — sécurité.

4.2.4 Analyser continuellement la qualité du code

Constat — Aucune analyse continue de la qualité, de la duplication ou de la dette technique n'est présente dans le pipeline.

Cause — La chaîne actuelle s'arrête aux tests et aux builds et ne dispose d'aucun contrôle qualité associé au commit.

Action possible — Intégrer l'analyse continue demandée par le projet, établir une baseline puis définir un contrôle d'acceptation centré sur les nouvelles anomalies. Le mode d'hébergement et les seuils seront comparés avant décision.

Bénéfices attendus — Suivre la dette, éviter les régressions de qualité et fournir un résultat compréhensible pendant les merge requests.

Priorité et périmètre — Élevée — P1 — qualité, sécurité et maintenabilité.

4.2.5 Automatiser et tracer la production des images

Constat — Les images sont construites et transmises manuellement ; aucune image publiée ne relie actuellement le code, les tests et les contrôles.

Cause — Le pipeline ne possède ni étape de publication, ni identifiant immuable, ni manifeste de livraison.

Action possible — Construire les images à partir des livrables validés, les publier avec une identité immuable et produire un manifeste reliant commit, pipeline, version et images.

Bénéfices attendus — Supprimer les écarts de version, faciliter les contrôles Ops et permettre de retrouver précisément l'origine d'une livraison.

Priorité et périmètre — Élevée — P1 — fiabilité et traçabilité.

4.2.6 Rendre la configuration d'exécution portable

Constat — Le frontend contient une URL localhost, le port déclaré par l'image backend est incohérent, la configuration Caddy empêche le smoke test attendu et aucun healthcheck n'est défini.

Cause — La configuration dépend du poste local et les contrats réseau ne sont pas

uniformisés entre l'application et les conteneurs.

Action possible — Externaliser les paramètres d'environnement, utiliser des routes ou noms de service stables, aligner les ports et le routage, puis ajouter des contrôles de santé et des smoke tests. Aucune adresse IP d'infrastructure ne devra être codée dans l'application.

Bénéfices attendus — Déployer la même version dans plusieurs environnements avec moins de modifications et détecter rapidement une configuration inutilisable.

Priorité et périmètre — Élevée — P1 — fiabilité et maintenabilité.

4.2.7 Définir la persistance, la sauvegarde et la restauration

Constat — La base HSQLDB actuelle perd les données lorsqu'un conteneur est remplacé, tandis que les sondages ne décrivent pas exactement la même cible de données.

Cause — L'environnement de démonstration privilégie une base embarquée et aucun besoin de persistance ou de restauration n'est formalisé.

Action possible — Clarifier les données à conserver et la cible d'environnement, puis définir un contrat de persistance, de sauvegarde et de restauration. Les scripts pourront être testés sans risque en P1 ; la preuve AWS réelle relèvera de P2.

Bénéfices attendus — Éviter une perte de données et disposer de critères vérifiables pour la reprise après incident.

Priorité et périmètre — Élevée — cadrage P1, exécution P2 — fiabilité.

4.2.8 Normaliser le workflow Git et les releases

Constat — Les règles de branches, de merge request, de versioning et de responsabilité de livraison ne sont pas documentées ; certaines références d'images ou d'outils restent flottantes.

Cause — Le dépôt fournit des commandes techniques mais pas de processus partagé reliant changement, validation et version livrable.

Action possible — Définir les événements du pipeline, les validations avant intégration, les conventions de version et l'identité technique des livrables ; maîtriser les versions utilisées par la chaîne.

Bénéfices attendus — Rendre les intégrations prévisibles, les releases reproductibles et les responsabilités compréhensibles par Dev et Ops.

Priorité et périmètre — Élevée — P1 — maintenabilité et fiabilité.

4.2.9 Modulariser les opérations répétitives

Constat — Plusieurs opérations de dépendances, tests, builds, contrôles et transmission sont répétées ou réalisées manuellement.

Cause — La logique est répartie entre les commandes locales, le YAML CI et les habitudes des équipes, sans interface commune.

Action possible — Créer de petits scripts par responsabilité, avec paramètres, codes retour, journaux et mode de simulation lorsque nécessaire, puis les appeler de la même façon en local et dans des templates CI réutilisables.

Bénéfices attendus — Réduire les divergences entre environnements, éviter la duplication et simplifier les tests et la maintenance de la chaîne.

Priorité et périmètre — Élevée — P1 — rapidité, maintenabilité et coût.

4.2.10 Automatiser le retour d'information utile

Constat — La livraison et les contrôles tardifs nécessitent des échanges manuels, mais aucun mécanisme ne synthétise actuellement le résultat du pipeline ou d'une release.

Cause — La chaîne ne centralise pas encore les statuts, rapports et références de livrables dans un message commun.

Action possible — Construire une notification standard à partir des informations GitLab et séparer ce message du canal de diffusion, qui sera choisi après comparaison des contraintes.

Bénéfices attendus — Informer rapidement les acteurs concernés sans multiplier les messages et faciliter l'accès au pipeline ou au rapport en cause.

Priorité et périmètre — Moyenne — P1 — rapidité et coordination.

5. Conclusion

5.1 Synthèse des constats

Récapituler les principaux constats de la veille technologique et de l'audit, ainsi que les recommandations proposées.

5.2 Prochaines étapes

Identifier les prochaines étapes pour la mise en œuvre des recommandations et proposer une trame de planification de celles-ci. Notez qu'il n'est pas nécessaire d'implémenter toutes ces recommandations dans le cadre de ce projet.